

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Теорія прийняття рішень»

Робота №2

Тема «Оптимальна оцінка значення прогнозованого параметра»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дейнега Л.Ю.

2025

МЕТА

Метою роботи є вивчити один із евристичних алгоритмів індивідуального прогнозування – класифікацію за однією ознакою з використанням даних навчального експерименту; навчитися обирати оптимальне порогове значення ознаки.

ЗАВДАННЯ

Завдання на роботу:

Використовуючи конспект лекцій та рекомендовану літературу, вивчити теорію статистичного оцінювання і її використання при розв’язанні задачі індивідуального прогнозування за ознаками з оцінкою значення прогнозованого параметра. Ознайомитися зі змістом і порядком виконання роботи.

ВИКОНАННЯ

Код

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar, newton

# --- 1. Вихідні дані ---
M_X = 10.0
M_Y = 20.0
SIGMA_X = 0.7
SIGMA_Y = 0.9
D_Y = SIGMA_Y**2 # 0.81
R1 = -0.7
R2 = -0.9
X_VALS = [12.1, 11.4, 10.0, 8.6] # X(1)...X(4)

# --- 2. Функції (Завдання 2.3.2) ---
```

```

def get_cond_stats(x, r, m_x=M_X, m_y=M_Y, s_x=SIGMA_X, s_y=SIGMA_Y):
    """Повертає умовне мат. сподівання та умовну дисперсію."""
    # M[y/x]
    m_cond = m_y + r * (s_y / s_x) * (x - m_x)
    # D[y/x] = S_y^2 * (1 - r^2)
    d_cond = (s_y**2) * (1 - r**2)
    return m_cond, d_cond

def gaussian_pdf(val, mean, var):
    """Одномірна густина нормального розподілу."""
    std = np.sqrt(var)
    return (1 / (std * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((val -
mean) / std) ** 2)

def bivariate_pdf(x, y, r, m_x=M_X, m_y=M_Y, s_x=SIGMA_X, s_y=SIGMA_Y):
    """Спільна густина розподілу W(x, y)."""
    coef = 1 / (2 * np.pi * s_x * s_y * np.sqrt(1 - r**2))
    z = (
        ((x - m_x) ** 2 / s_x**2)
        - (2 * r * (x - m_x) * (y - m_y) / (s_x * s_y))
        + ((y - m_y) ** 2 / s_y**2)
    )
    return coef * np.exp(-z / (2 * (1 - r**2)))

# --- 3. Завдання 2.3.3: Залежність дисперсії похибки від r та D[y] ---
print(f"{'=' * 20} ЗАВДАННЯ 2.3.3 {'=' * 20}")
r_range = [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
dy_multipliers = [1, 2, 4] # D[y], 2*D[y], 4*D[y]

print(f"{'|r|':<5} | {'D_err (Dy)':<12} | {'D_err (2Dy)':<12} | {'D_err (4Dy)':<12}")
print("-" * 50)

results_233 = {m: [] for m in dy_multipliers}

for r in r_range:
    row = [f"{r:<5}"]
    for mult in dy_multipliers:
        current_dy = D_Y * mult
        # D_error = Dy * (1 - r^2)
        d_error = current_dy * (1 - r**2)
        results_233[mult].append(d_error)
        row.append(f"{d_error:<12.4f}")
    print(" | ".join(row))

# Графік для 2.3.3
plt.figure(figsize=(8, 5))
for mult in dy_multipliers:
    plt.plot(r_range, results_233[mult], marker="o", label=f"D[y] * {mult}")
plt.title("Залежність дисперсії похибки прогнозу від коефіцієнта кореляції")
plt.xlabel("|r|")
plt.ylabel("Дисперсія похибки D[y|x]")
plt.grid(True)

```

```

plt.legend()
plt.show()

# --- 4. Завдання 2.3.4: Безумовна густина W(y) ---
print(f"\n{'=' * 20} ЗАВДАННЯ 2.3.4 (Безумовна W(y)) {'=' * 20}")
# Точки: My, My +/- k*Sigma
sigmas_steps = [0, 0.2, 0.4, 0.6, 0.8, 1.0, 2.0, 3.0]
# Генеруємо список точок y (симетрично)
y_points_check = []
for s in sigmas_steps:
    if s == 0:
        y_points_check.append(M_Y)
    else:
        y_points_check.append(M_Y - s * SIGMA_Y)
        y_points_check.append(M_Y + s * SIGMA_Y)
y_points_check = sorted(list(set(y_points_check)))

print(f"{'Y':<10} | {'W(y)':<10}")
print("-" * 25)
for y_val in y_points_check:
    w_val = gaussian_pdf(y_val, M_Y, SIGMA_Y**2)
    print(f"{y_val:<10.2f} | {w_val:<10.4f}")

# --- 5. Завдання 2.3.5 & 2.3.6: Умовні густини W(y/x) ---
print(f"\n{'=' * 20} ЗАВДАННЯ 2.3.5 (Умовні W(y|x)) {'=' * 20}")
# Розрахунок для R1 та R2, та для всіх X
correlations = [R1, R2]
plt.figure(figsize=(12, 6))

# Побудова графіків (Завдання 2.3.6)
# Спочатку малюємо безумовну W(y) для порівняння
y_axis = np.linspace(M_Y - 4 * SIGMA_Y, M_Y + 4 * SIGMA_Y, 200)
plt.plot(
    y_axis,
    gaussian_pdf(y_axis, M_Y, SIGMA_Y**2),
    "k--",
    linewidth=2,
    label="Безумовна W(y)",
)

colors = ["b", "g", "r", "c"] # для різних X

for r_idx, r_val in enumerate(correlations):
    print(f"\n--- Кореляція r = {r_val} ---")
    print(f"{'X(j)':<6} | {'M[y|x]':<8} | {'Sigma[y|x]':<10} | {'D[y|
x]':<8}")

    for i, x_val in enumerate(X_VALS):
        m_cond, d_cond = get_cond_stats(x_val, r_val)
        s_cond = np.sqrt(d_cond)

        print(f"{x_val:<6.1f} | {m_cond:<8.2f} | {s_cond:<10.3f} |
{d_cond:<8.3f}")

        # Графік умовної щільності
        if (
            r_val == R1

```

```

        ): # Малюємо тільки для R1 щоб не перевантажувати графік, або
        можна розділити
            label = f"W(y|x={x_val}), r={r_val}"
            plt.plot(
                y_axis,
                gaussian_pdf(y_axis, m_cond, d_cond),
                color=colors[i],
                label=label,
            )

plt.title(f"Умовні густини розподілу (для r={R1}) порівняно з
безумовною")
plt.xlabel("Y")
plt.ylabel("W")
plt.legend()
plt.grid(True)
plt.show()

# --- 6. Завдання 2.3.7, 2.3.8, 2.3.9: Оптимізація ---
print(f"\n{'=' * 20} ЗАВДАННЯ 2.3.7 - 2.3.9 (Оптимізація) {'=' * 20}")

# Вибираємо X(2) згідно завдання
X_target = X_VALS[1] # 11.4
R_target = R1 # -0.7

# Функція для мінімізації (мінус густина), оскільки методи шукають
мінімум
# Ми шукаємо максимум W(y|x) по Y при фіксованому X
def func_to_minimize(y):
    # Використовуємо спільну густину, бо максимізація W(x,y) по y
    еквівалентна максимізації W(y|x)
    return -1 * bivariate_pdf(X_target, y, R_target)

# Точне аналітичне рішення (Завдання 2.3.8)
true_optimal_y, _ = get_cond_stats(X_target, R_target)
print(f"Аналітичне рішення (точне M[y|x]): {true_optimal_y:.6f}")

# 1. Метод розподілу інтервалу навпіл (Bisection) - для пошуку нуля
похідної
# Або можна використати `minimize_scalar` з методом 'bounded' як аналог
res_bisection = minimize_scalar(func_to_minimize, bounds=(10, 30),
method="bounded")
y_bisect = res_bisection.x

# 2. Метод Пауела (Powell)
res_powell = minimize_scalar(
    func_to_minimize, method="brent"
) # Brent - це покращений метод, схожий на Powell для 1D
# Для строгого слідування назві "Powell" треба використовувати
scipy.optimize.minimize для багатовимірних,
# але для 1D це надлишково. Використаємо 'Golden' або 'Brent' як
стандартні 1D методи.
# Але якщо потрібно саме ітеративний пошук мінімуму:
from scipy.optimize import minimize

res_powell_real = minimize(func_to_minimize, x0=20.0, method="Powell")

```

```

y_powell = res_powell_real.x[0]

# 3. Метод Ньютона-Рафсона (Newton-Raphson)
# Ньютон шукає корінь рівняння  $f(x)=0$ . Нам потрібен корінь ПОХІДНОЇ
# густини.
# Похідна  $W(x,y)$  по  $y$  дорівнює 0, коли  $y = M[y|x]$ .
# Визначимо похідну аналітично або чисельно. Для Ньютона зручніше взяти
# аналітичне значення.
#  $f'(y) \sim -(y - M_{\text{cond}})$ . Корінь очевидний, але запустимо метод.
def derivative_func(y):
    # Повертає наближене значення похідної (різницева схема)
    h = 1e-5
    return (func_to_minimize(y + h) - func_to_minimize(y)) / h

y_newton = newton(derivative_func, x0=19.0)

print("-" * 60)
print(
    f"{'Метод':<20} | {'Значення Y*':<15} | {'Абс. похибка':<15} | "
    {'Відн. похибка (%)':<15}"
)
print("-" * 60)

methods = {
    "Analytic (2.3.8)": true_optimal_y,
    "Bisection/Bounded": y_bisect,
    "Powell": y_powell,
    "Newton-Raphson": y_newton,
}

for name, val in methods.items():
    abs_err = abs(val - true_optimal_y)
    rel_err = (abs_err / true_optimal_y) * 100 if true_optimal_y != 0
    else 0
    print(f"{name:<20} | {val:<15.6f} | {abs_err:<15.2e} | "
          {rel_err:<15.4e}")

```

Результати

===== ЗАВДАННЯ 2.3.3 =====			
r	D_err (Dy)	D_err (2Dy)	D_err (4Dy)
0	0.8100	1.6200	3.2400
0.1	0.8019	1.6038	3.2076
0.2	0.7776	1.5552	3.1104
0.3	0.7371	1.4742	2.9484
0.4	0.6804	1.3608	2.7216
0.5	0.6075	1.2150	2.4300
0.6	0.5184	1.0368	2.0736
0.7	0.4131	0.8262	1.6524
0.8	0.2916	0.5832	1.1664
0.9	0.1539	0.3078	0.6156
1.0	0.0000	0.0000	0.0000

===== ЗАВДАННЯ 2.3.4 (Безумовна W(y)) =====	
Y	W(y)
17.30	0.0049
18.20	0.0600
19.10	0.2689
19.28	0.3219
19.46	0.3702
19.64	0.4092
19.82	0.4345
20.00	0.4433
20.18	0.4345
20.36	0.4092
20.54	0.3702
20.72	0.3219
20.90	0.2689
21.80	0.0600
22.70	0.0049

Рисунок 1.1 — Перші таблиці

===== ЗАВДАННЯ 2.3.5 (Умовні $W(y|x)$) =====

--- Кореляція $r = -0.7$ ---

$X(j)$	$M[y x]$	$\text{Sigma}[y x]$	$D[y x]$
12.1	18.11	0.643	0.413
11.4	18.74	0.643	0.413
10.0	20.00	0.643	0.413
8.6	21.26	0.643	0.413

--- Кореляція $r = -0.9$ ---

$X(j)$	$M[y x]$	$\text{Sigma}[y x]$	$D[y x]$
12.1	17.57	0.392	0.154
11.4	18.38	0.392	0.154
10.0	20.00	0.392	0.154
8.6	21.62	0.392	0.154

===== ЗАВДАННЯ 2.3.7 - 2.3.9 (Оптимізація) =====

Аналітичне рішення (точне $M[y|x]$): 18.740000

Метод	Значення Y^*	Абс. похибка	Відн. похибка (%)
Analytic (2.3.8)	18.740000	0.00e+00	0.0000e+00
Bisection/Bounded	18.740000	3.76e-08	2.0061e-07
Powell	18.740000	1.76e-07	9.3847e-07
Newton-Raphson	18.739995	5.00e-06	2.6681e-05

Рисунок 1.2 — Другі таблиці

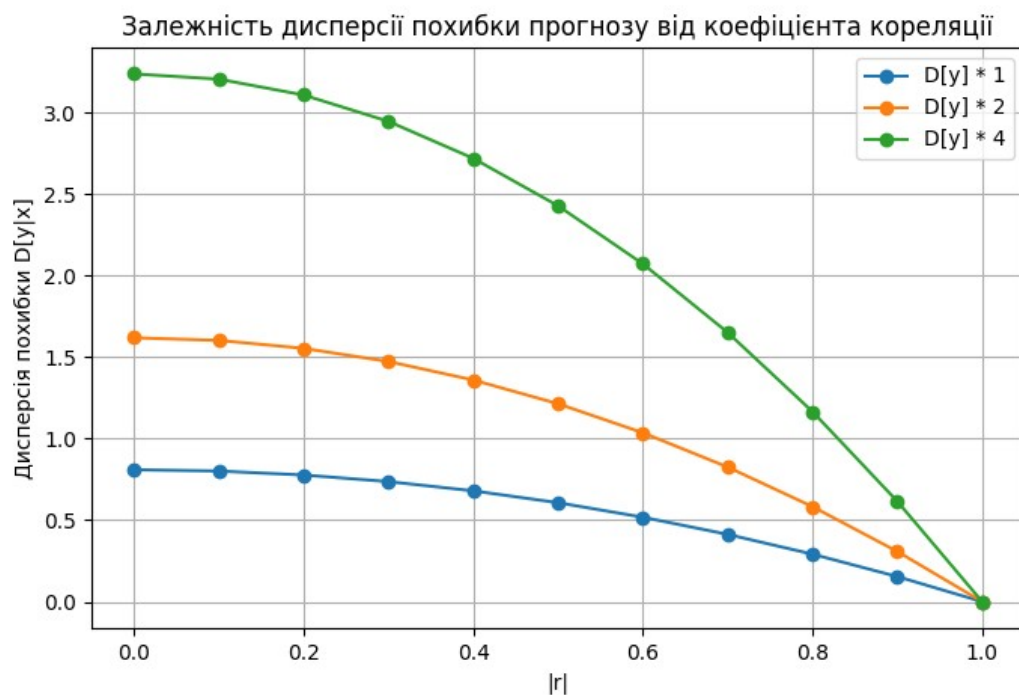


Рисунок 1.3 — Залежності дисперсій

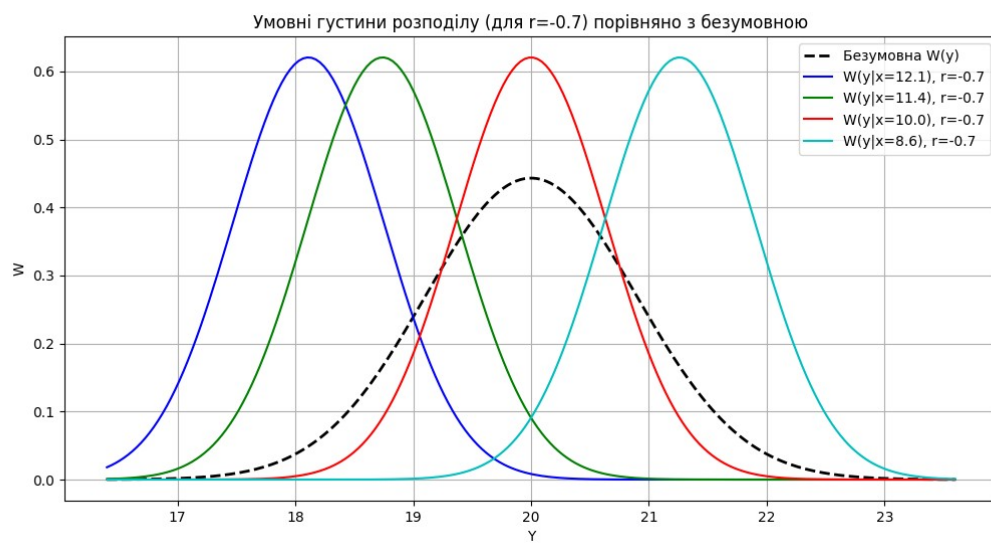


Рисунок 1.4 — Умовні густини розподілу